



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

Лабораторная работа № 7
по курсу «Языки и методы программирования»
«GUI-чат на Java Swing и MQTT»

Студент группы ИУ9-22Б
Булдаков А. С.

Преподаватель
Посевин Д. П.

Москва 2026

Содержание

Цель работы	1
Теоретические сведения	1
Библиотека FlatLaF	1
Архитектура приложения	1
Реализация приложения	2
Класс ChatGUI	2
Инициализация компонентов	3
Подключение к брокеру	5
Отправка сообщений	7
Главный метод	8
Сборка и запуск	8
Используемые библиотеки	8
Makefile	8
Компиляция и запуск	9
Вывод	9

Цель работы

Разработать графический чат-клиент на Java с использованием библиотек Swing и Eclipse Paho MQTT. Приложение должно обеспечивать визуальный интерфейс для отправки и получения сообщений через MQTT-брокер.

Задачи:

- Изучить компоненты библиотеки Swing для создания GUI.
- Создать окно чата с полем ввода сообщений и списком доступных топиков.
- Реализовать подключение к MQTT-брокеру для отправки и получения сообщений.
- Использовать библиотеку FlatLaF для современного внешнего вида.

Теоретические сведения

Библиотека FlatLaF

FlatLaF — это библиотека для создания современного пользовательского интерфейса в Swing-приложениях. Она предоставляет различные темы оформления:

- FlatDarkLaf — тёмная тема
- FlatMacLightLaf — светлая тема в стиле macOS

Архитектура приложения

Приложение ChatGUI состоит из следующих компонентов:

- Главное окно (JFrame)
- Панель настроек (топик, имя пользователя)
- Область сообщений (JTextArea)
- Панель ввода сообщений (JTextField, JButton)
- Список доступных топиков (JTextArea)

Реализация приложения

Класс ChatGUI

ChatGUI.java

```
public class ChatGUI extends JFrame {
    private JTextField topicInput;
    private JTextField usernameInput;
    private JTextArea messagesArea;
    private JTextField messageInput;
    private JButton sendButton;
    private JTextArea topicsArea;

    private MqttClient client;
    private MqttClient topicClient;

    public ChatGUI() {
        setTitle("MQTT Chat");
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setSize(600, 500);
        setLayout(new BorderLayout(10, 10));

        initComponents();
        connectToBroker();
    }
}
```

Инициализация компонентов

```
private void initComponents() {
    JPanel topPanel = new JPanel(new BorderLayout(5, 5));
    topPanel.setBorder(BorderFactory.createEmptyBorder(10, 10, 10,
10));

    JPanel topicsPanel = new JPanel(new BorderLayout(5, 5));
    topicsPanel.add(new JLabel("Доступные топики:"),
BorderLayout.NORTH);

    topicsArea = new JTextArea(3, 30);
    topicsArea.setEditable(false);
    topicsPanel.add(new JScrollPane(topicsArea),
BorderLayout.CENTER);

    JPanel inputPanel = new JPanel(new GridLayout(2, 2, 5, 5));
    inputPanel.add(new JLabel("Топик:"));
    topicInput = new JTextField(currentTopic);
    inputPanel.add(topicInput);

    inputPanel.add(new JLabel("Имя:"));
    usernameInput = new JTextField("");
    inputPanel.add(usernameInput);

    topPanel.add(topicsPanel, BorderLayout.NORTH);
    topPanel.add(inputPanel, BorderLayout.CENTER);

    messagesArea = new JTextArea();
    messagesArea.setEditable(false);
    messagesArea.setLineWrap(true);
    messagesArea.setWrapStyleWord(true);
    JScrollPane messagesScroll = new JScrollPane(messagesArea);

    messagesScroll.setBorder(BorderFactory.createTitledBorder("Сообщения"));

    JPanel bottomPanel = new JPanel(new BorderLayout(5, 5));
    messageInput = new JTextField();
    messageInput.addActionListener(e → sendMessage());

    sendButton = new JButton("Отправить");
    sendButton.addActionListener(e → sendMessage());

    bottomPanel.add(messageInput, BorderLayout.CENTER);
    bottomPanel.add(sendButton, BorderLayout.EAST);

    add(topPanel, BorderLayout.NORTH);
    add(messagesScroll, BorderLayout.CENTER);
    add(bottomPanel, BorderLayout.SOUTH);
}
```

Подключение к брокеру

```

private void connectToBroker() {
    String broker = "tcp://localhost:1883";

    try {
        client = new MqttClient(broker, clientId, new
MemoryPersistence());
        client.setCallback(new MqttCallback() {
            @Override
            public void messageArrived(String topic, MqttMessage message)
throws Exception {
                String msg = new String(message.getPayload());
                SwingUtilities.invokeLater(() → {
                    messagesArea.append(msg + "\n");

messagesArea.setCaretPosition(messagesArea.getDocument().getLength());
                });
            }

            @Override
            public void deliveryComplete(IMqttDeliveryToken token) {
            }

            @Override
            public void connectionLost(Throwable cause) {
                SwingUtilities.invokeLater(() → {
                    JOptionPane.showMessageDialog(ChatGUI.this,
                        "Соединение потеряно!", "Ошибка",
JOptionPane.ERROR_MESSAGE);
                });
            }
        });

        client.connect();
        client.subscribe(currentTopic, 1);

        // второй клиент для отслеживания всех топиков
        topicClient = new MqttClient(broker, topicClientId, new
MemoryPersistence());
        topicClient.setCallback(new MqttCallback() {
            @Override
            public void messageArrived(String topic, MqttMessage message)
throws Exception {
                String msg = new String(message.getPayload());
                SwingUtilities.invokeLater(() → {
                    if (!allChats.contains(topic)) {
                        allChats.add(topic);
                        topicsArea.append(topic + "\n");
                    }
                });
            }
        });
    }
}

```

Отправка сообщений

ChatGUI.java

```
private void sendMessage() {
    String message = messageInput.getText().trim();
    if (message.isEmpty())
        return;

    String username = usernameInput.getText().trim();
    if (username.isEmpty()) {
        username = "User";
    }

    String newTopic = topicInput.getText().trim();
    boolean topicChanged = false;
    if (!newTopic.equals(currentTopic)) {
        try {
            if (client != null && client.isConnected()) {
                client.unsubscribe(currentTopic);
                client.subscribe(newTopic, 1);
                currentTopic = newTopic;
                topicChanged = true;
            }
        } catch (MqttException e) {
            JOptionPane.showMessageDialog(this, "Ошибка смены топика: " +
e.getMessage());
        }
    }

    if (topicChanged) {
        messagesArea.setText("");
        appendMessage("≡ Подключено к новому топик: " + currentTopic
+ " ≡");
    }

    String timestamp = LocalDateTime.now().format(formatter);
    String fullMessage = "[" + timestamp + "] " + username + ": " +
message;

    try {
        client.publish(currentTopic, new
MqttMessage(fullMessage.getBytes()));
        messageInput.setText("");
    } catch (MqttException e) {
        JOptionPane.showMessageDialog(this, "Ошибка отправки: " +
e.getMessage());
    }
}
```


Главный метод

ChatGUI.java

```
public static void main(String[] args) {
    SwingUtilities.invokeLater(() → {
        try {
            UIManager.setLookAndFeel(new FlatDarkLaf());
        } catch (Exception e) {
            e.printStackTrace();
        }
        new ChatGUI().setVisible(true);
    });
}
```

Сборка и запуск

Используемые библиотеки

terminal

```
$ ls -la lib/
-rw-r--r-- 1 root ... flatlaf-3.7.jar
-rw-r--r-- 1 root ... org.eclipse.paho.client.mqttv3-1.2.5.jar
```

Makefile

Makefile

```
.PHONY: all clean run

all:
    javac -cp lib/flatlaf-3.7.jar:lib/
    org.eclipse.paho.client.mqttv3-1.2.5.jar \
        -d out src/ChatGUI.java

run:
    java -cp out:lib/flatlaf-3.7.jar:lib/
    org.eclipse.paho.client.mqttv3-1.2.5.jar \
        ChatGUI

clean:
    rm -f out/*.class
```

Компиляция и запуск

```
terminal
$ make
$ make run
```

После запуска открывается окно чата с:

- Полем ввода топика
- Полем ввода имени пользователя
- Областью сообщений
- Полем ввода сообщения и кнопкой «Отправить»
- Списком доступных топиков

Вывод

В ходе лабораторной работы был разработан GUI-чат на Java Swing с использованием протокола MQTT. Приложение ChatGUI обеспечивает:

- Графический интерфейс пользователя с использованием библиотеки FlatLaF
- Подключение к MQTT-брокеру для обмена сообщениями
- Возможность выбора топика для чата
- Отображение списка доступных топиков
- Добавление временной метки к сообщениям
- Автоматическую смену топика без переподключения

Использование Swing и MQTT позволяет создавать современные приложения для обмена сообщениями в реальном времени.